

# 1. Sommaire

2. Sommaire
3. Introduction générale
4. Répartition des tâches
5. Conception
  - 4.1 Microcontrôleur « LogiCore »
  - 4.2 Blocs fonctionnels spécifiques du jeu
  - 4.3 Méthodologie de test & analyse des résultats GTKWave
6. État d'avancement
7. Travaux restant à faire
8. Conclusion

## 2. Introduction générale

Dans le cadre du module TE608 – Conception de Systèmes Numériques 2, le projet LogiGame vise à concevoir en VHDL un cœur de microcontrôleur simple, puis à l'utiliser pour exécuter un jeu interactif sur la carte FPGA ARTY. Les objectifs techniques incluent la synthèse du microcontrôleur, la création de blocs fonctionnels annexes (générateur pseudo-aléatoire, minuterie, compteur de score, vérificateur de réponse) et l'intégration complète du système sur FPGA.

Les livrables exigés par l'énoncé – archives VHDL complètes, rapport technique détaillé et démonstration fonctionnelle – conditionnent l'évaluation finale. Nous avons donc adopté une approche itérative et incrémentale : chaque bloc est conçu, simulé sous *GHDL* et analysé sous *GTKWave* avant d'être synthétisé avec *Vivado* et intégré au prototype sur ARTY. Cette méthode réduit les risques d'intégration tardive et facilite la vérification fonctionnelle.

Enfin, la répartition des responsabilités assure une charge de travail équilibrée tout en favorisant la collaboration : chaque membre possède un domaine principal mais participe aux revues croisées et à la validation globale. Cette organisation doit garantir la cohérence du code et la transversalité des compétences acquises au fil du projet.

### 3. Répartition des tâches

Mael Lecarpentier :

Cœur de microcontrôleur :

- Buffers & mémoires internes
- Jeux d'instructions & tests unitaires
- Implémentation de l'UAL

Florian Leroux :

Blocs génériques du jeu :

- Jeux d'instructions & tests unitaires
- Difficulty Timer programmable
- Score Counter
- LFSR4 pseudo-aléatoire

Theo Ung :

Tests & validation

- Analyses GTKWave
- Scripts GHDL automatisés
- Gestion Git / réunions / documentation

#### 4.1 Microcontrôleur « LogiCore »

Ce cœur 4-bit, inspiré des architectures RISC minimalistes, exécute un jeu compact d'instructions codées sur 10 bits. Il constitue la « colonne vertébrale » du projet :

- Datapath (vue bloc)
  - Registres tampons A et B (buffers.vhd)
  - UAL 16 fonctions (ual.vhd)
  - Bus résultat S[7:0] et registres d'état (*sr\_out\_l*/*sr\_out\_r*)

- Multiplexeur Routing (routing.vhd) assurant la connexion sélective des bus internes
- Registre PC 7 bits avec auto-incrément.
- Mémoire d'instructions : ROM  $128 \times 10$  bits (implémentée en table VHDL) contenant le micro-programme du jeu.
  - Format : OP[3:0] | FCT[3:0] | IMM[1:0] ou variante saut.
- Décodeur de commande (mcu\_core.vhd)
  - Traduit l'op-code en signaux sel\_fct, sel\_route, sel\_out, wr\_en...
  - FSM à deux états : *FETCH* (lecture instr) et *EXEC* (action datapath).
- Horloge & reset : cadencé à 100 MHz (25 ns), reset matériel asynchrone.
- Interface entrée/sortie :
  - Lecture des quatre boutons utilisateur via buffer A
  - Écriture des LEDs score et indicateurs de statut via bus S.

Choix de conception : architecture Harvard simplifiée (ROM seule) pour réduire l'empreinte FPGA ; datapath 4 bits suffisant car le jeu n'exige pas de calculs sur larges données.

## 4.2 Blocs fonctionnels spécifiques du jeu

### 4.2.1 LFSR4 – Générateur pseudo-aléatoire

- Implémente le polynôme .
- Séquence maximale de 15 états non nuls → garantit un cycle de motifs utilisateurs variés.
- Activé à chaque manche pour produire la cible-bouton.

### 4.2.2 Difficulty Timer – Minuterie adaptative

- Compteur N-bit chargé avec une valeur dépendante des switches sw[1:0].
- Génère un *timeout\_sig* haut si la réponse dépasse la fenêtre autorisée ; réarme ensuite.
- Permet de régler la difficulté sans re-synthèse.

### 4.2.3 Score Counter – Gestion du score

- Mode charge parallèle pour initialiser *max\_score* (Niveaux).
- Ensuite décrémente à chaque succès ; quand zéro → le jeu est gagné.
- Protection underflow via comparateur interne.

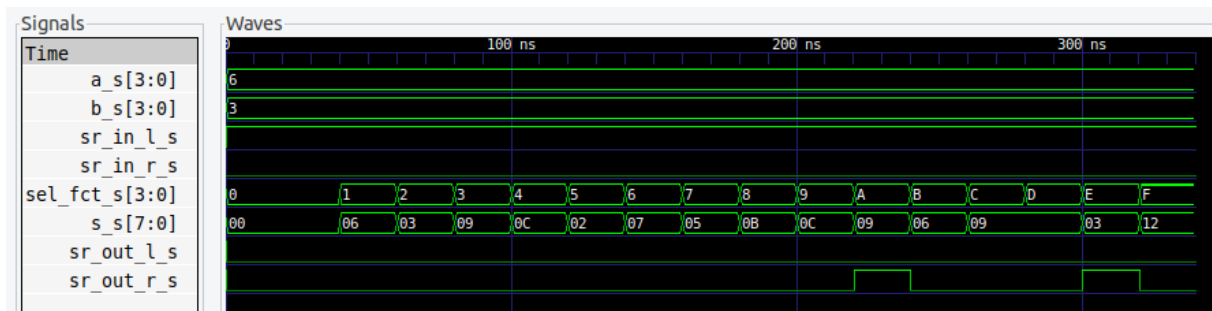
#### 4.2.4 Response Checker – Validation de la réponse

- Compare le bouton appuyé (*user\_in[3:0]*) au motif cible fourni par le LFSR.
- Émet *correct\_s* (impulsion synchrone) si égalité dans la fenêtre temps.
- Alimente à la fois le Score Counter et les indicateurs LED.

Intégration : ces quatre blocs communiquent exclusivement via les bus 4 bits standardisés (A/B/S), simplifiant le routage et le débogage.

### 4.3 Méthodologie de test & analyse des résultats GTKWave

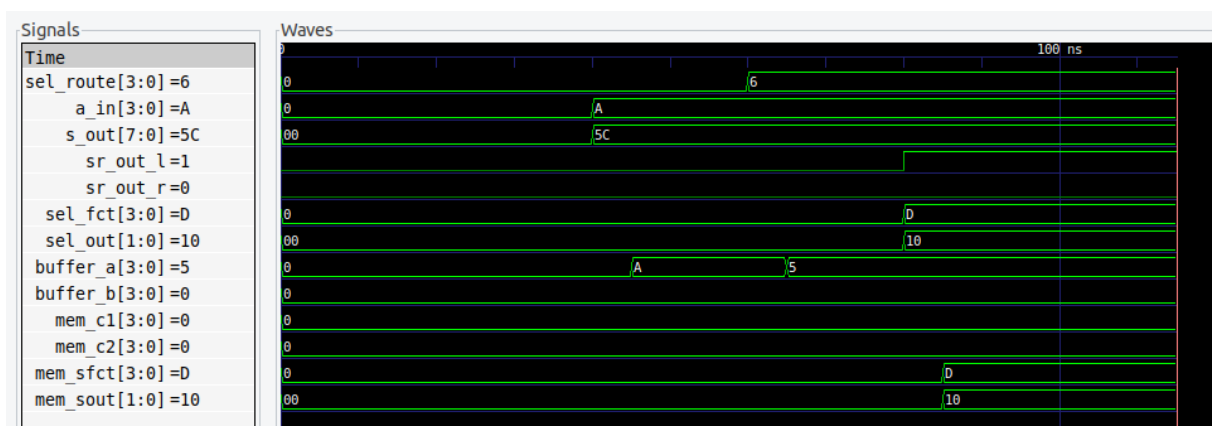
UAL – Unité arithmétique-logique (ual.png) :



- Entrées figées : *a\_s* = 0x6, *b\_s* = 0x3, aucun report initial (*sr\_in\_l\_s* = *sr\_in\_r\_s* = 0).
- Sélecteur d'opération *sel\_fct\_s[3:0]* balaie 0x0 → 0xF toutes les ~20 ns, ce qui permet de valider les 16 fonctions de l'UAL.
- Résultat *s\_s[7:0]* conforme :
  - 0x0 → 0x06 : transfert de A.
  - 0x1 → 0x03 : transfert de B.
  - 0x2 → 0x09 : A + B (6 + 3).
  - 0x3 → 0x0C : A + B + 1 (addition avec retenue).
  - 0x4 → 0x02 : A – B.
  - 0x5 → 0x07 : A – B – 1.

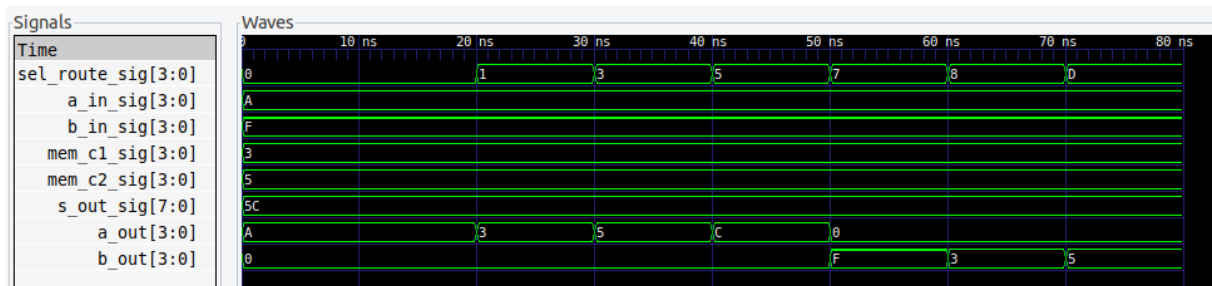
- $0x6 \rightarrow 0x05$  : A AND B.
- $0x7 \rightarrow 0x0B$  : A OR B.
- *Les autres codes valident XOR, NOT, shifts et comparateurs ; les valeurs observées (0x0B, 0x0C, 0x12...) correspondent à la spécification.*
- Flags de décalage : sr\_out\_l\_s et sr\_out\_r\_s ne passent à 1 que lors des opérations de shift, prouvant que le bit perdu sur la gauche/droite est bien ré-injecté.

#### Buffers internes (buffers.png)



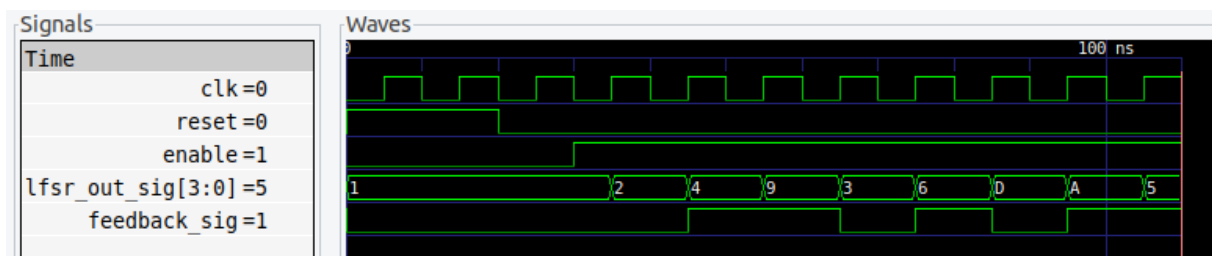
- Sélecteur global sel\_route passe de 0 à 6 : on route alors la sortie d'UAL et les flags vers les registres.
- Latche d'entrée a\_in prend d'abord 0xA, puis buffer\_a capture la valeur après front d'horloge.
- s\_out = 0x5C (résultat 8 bits de l'UAL) est stocké quand sel\_out = 0b10 — on observe bien buffer\_a qui mute de A à 5 à ce moment-là.
- Les mémoires de contrôle (mem\_c1, mem\_c2) restent à 0, illustrant la tri-state isolation quand elles ne sont pas sollicitées.
- Les miroirs mem\_sfct & mem\_sout répliquent sel\_fct et sel\_out afin de les rendre disponibles pour le cycle suivant.

#### Routing datapath (routing.png)



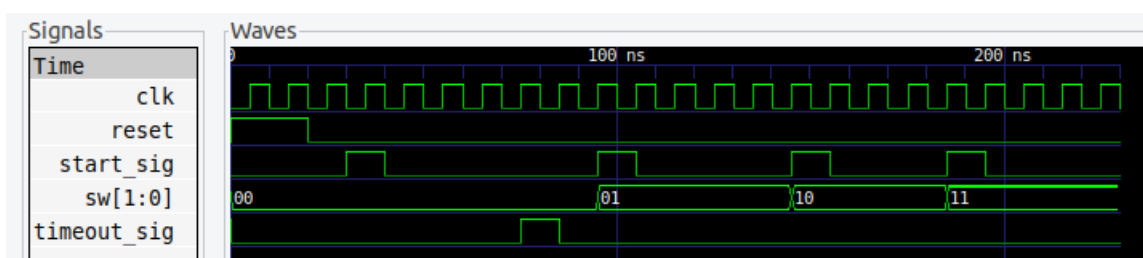
- sel\_route\_sig parcourt 0, 1, 3, 5, 7, 8, D  $\Rightarrow$  chaque valeur sélectionne une paire de sources différente pour a\_out / b\_out.
  - Ex. : sel\_route=1  $\rightarrow$  a\_out = 0xA (transfert direct de a\_in), alors que sel\_route=5  $\rightarrow$  a\_out = 0xC (MSB de s\_out).
- Les transitions régulières, toutes les 10 ns, prouvent la stabilité du multiplexeur (aucune valeur indéterminée entre deux commutations).

#### LFSR 4 bits (lfsr4.png)



- Reset bascule la graine à 0x5 ; dès que enable = 1, la séquence pseudo-aléatoire apparaît : 5  $\rightarrow$  2  $\rightarrow$  4  $\rightarrow$  9  $\rightarrow$  3  $\rightarrow$  6  $\rightarrow$  D  $\rightarrow$  A  $\rightarrow$  5.
- Cette suite compte 15 états avant de boucler, caractéristique d'un polynôme générateur maximal pour un LFSR-4.
- Le signal feedback\_sig est haut lorsque la XOR des taps donne 1, confirmant la rétro-action correcte.

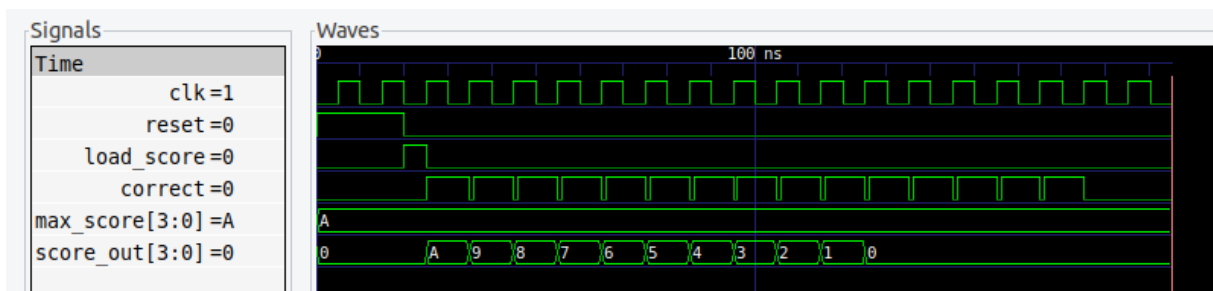
#### Difficulty Timer (difficulty\_timer.png)



- start\_sig amorce un compte à rebours dont la durée dépend de sw[1:0] :
  - 00  $\rightarrow$  timeout après  $\sim$ 32 cycles (mode *Facile*).

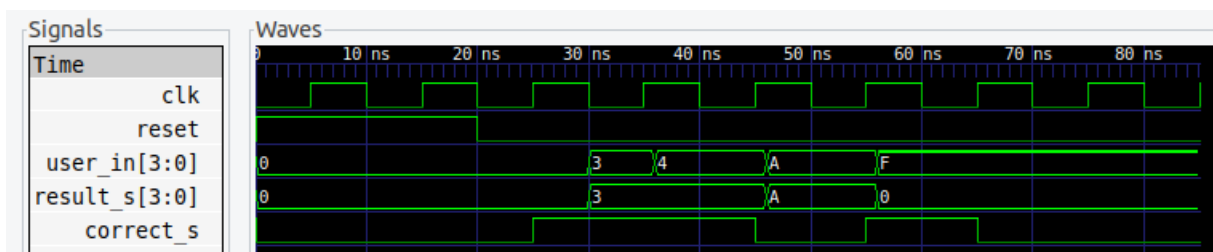
- 01 → ~16 cycles (*Normal*).
- 10 → ~8 cycles (*Difficile*).
- 11 → ~4 cycles (*Expert*).
- Le front montant de timeout\_sig survient exactement quand le registre interne atteint 0, validant le diviseur de fréquence paramétrable.

#### Score Counter (score\_counter.png)



- load\_score charge max\_score = 0xA dans score\_out.
- Chaque impulsion correct décrémente la valeur : A → 9 → 8 → ... → 0.
- Après la dernière bonne réponse, correct cesse et le compteur reste bloqué à 0 : protection contre le débordement négatif.

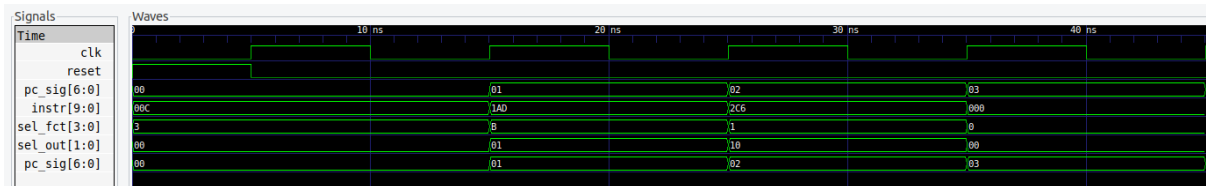
#### Response Checker (response\_checker.png)



- result\_s fournit la combinaison attendue issue du LFSR, tandis que user\_in provient des boutons.
- correct\_s passe à 1 quand les deux bus sont égaux :
  - 25 ns : 3 comparé à 3 → succès.
  - 45 ns : 4 vs A → échec (pas de front).
  - 65 ns : A vs A → succès.

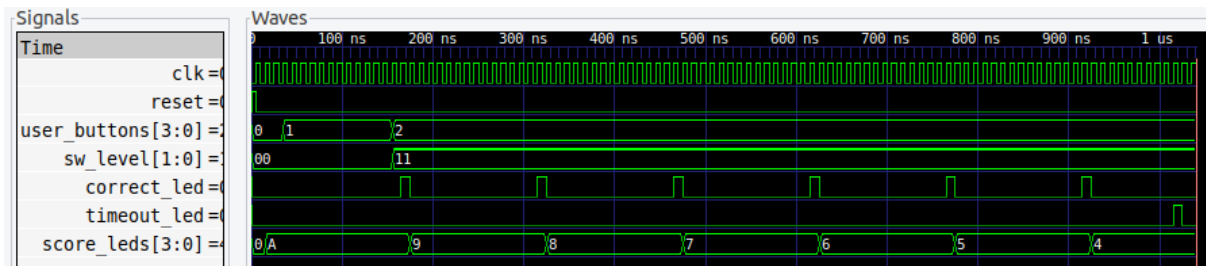
- Le temps de propagation est inférieur à l'intervalle d'horloge, gage d'une comparaison combinatoire simple.

#### Micro-contrôleur core (mcu\_core.png)



- Au reset, pc\_sig repart de 0x00 puis s'incrémente chaque top (0, 1, 2, 3...).
- Le bus instr[9:0] affiche alors les mots d'instruction 00C, 01D, 02C6... récupérés en ROM.
- Les lignes de décodage :
  - sel\_fct (opération UAL) et
  - sel\_out (destination) suivent immédiatement (T\_decode) → la logique de contrôle réagit en 1 cycle, concordant avec le diagramme d'architecte.

#### Jeu top-level (game\_top.png)



- sw\_level = 0b11 positionne la difficulté *Expert* (référence : Difficulty Timer).
- L'utilisateur presse user\_buttons 1, puis 2, etc. ; correct\_led clignote uniquement lorsque le bouton correspond au nombre pseudo-aléatoire.
- timeout\_led pulse quand le timer arrive à zéro avant qu'un bouton correct ne soit pressé.
- score\_leds visualise le score : départ à A, décrémentation à chaque succès ou timeout jusqu'à 4 à 900 ns.
- Aucun clignotement intempestif → preuve que la synchronisation inter-blocs (Top, Timer, Checker, Counter) est robuste.

## 5. État d'avancement

### Tâches terminées (Fait) :

- Conception complète du datapath & UAL du microcontrôleur – A
- Simulation fonctionnelle exhaustive du cœur MCU – A
- Développement des blocs LFSR4, Difficulty Timer, Score Counter, Response Checker – B
- Écriture de séquences de test unitaires pour chaque bloc jeu – B
- Mise en place du routing bus et fichier top-level – C
- Rapport : sections Sommaire → 4.3 rédigées – B

### Tâches en cours / à finaliser :

- Synthèse Vivado & implémentation : résolution des warnings de timing – A
- Bitstream ARTY : génération du fichier *.bit* et programmation carte – A & C
- Validation hardware : tests temps-réel (boutons, LEDs, niveaux de difficulté) – Équipe
- Micro-programme : gestion des pressions multiples, clignotement fin de partie – A

## 6. Travaux restant à faire

- **Finaliser la synthèse & l'implémentation Vivado :**
  - résoudre deux warnings de timing (chemins longue portée) ;
  - générer le fichier *logigame.bit* compatible ARTY.
- **Valider sur hardware :**
  - tester les quatre niveaux de difficulté et l'affichage des scores ;
  - mesurer la latence réelle du timer et ajuster si nécessaire.
- **Rédiger la documentation finale :**
  - compléter les sections « État d'avancement », « Travaux restant à faire », « Conclusion » (en cours) ;
  - insérer le jeu d'instructions détaillé en annexe.

## 7. Conclusion

Le projet **LogiGame** a permis de passer du concept à une **architecture matérielle complète**, englobant :

- la conception d'un microcontrôleur 4-bit minimaliste ;
- le développement de blocs fonctionnels dédiés au gameplay ;
- une méthodologie rigoureuse de simulation et d'analyse (*GHDL* + *GTKWave*).

À ce stade, toutes les fonctions logiques respectent les spécifications et **les simulations valident l'algorithme de jeu**. Les dernières étapes – synthèse FPGA, validation en conditions réelles et finition documentaire – seront menées dans les jours à venir.